Project 3: Distributed Systems (2PC & Raft) Report

1. Team Information

- Name: Pranav Pujar
- Student ID: 1001965075

- Name: Hanumath Ponnaluri
- Student ID: 1001866533

- Contribution Breakdown:

Pranav: Raft algorithm Implementation, ReadMe file

Hanumath: Two-Pc implementation, Word Report

2. Introduction

Describe the goal of the project and what consensus problems (2PC and Raft) solve.

The goal of the project was to choose another team's implementation from the 3 bids on the excel spreadsheet and implement a 2PC and raft algorithm extension to the existing project. So, our group chose to extend to group 11's implementation. Group 11 implemented a distributed fishing game system which includes both a single-node version and multi node 6 container Docker setup. This allows clients to connect, perform actions, and simulate a distributed environment. So our goal was to add a 2pc and raft algorithm to this project.

3. Design of 2PC Implementation

2PC algorithm was implemented using a coordinator-participant model to make sure consistency across the distributed nodes. The coordinator is responsible for managing the transaction lifecycle, while the participant nodes execute operations and vote on whether they can commit.

Communication between the node is done through gRPC-based RPCS which include key calls such as Prepare, Vote, and Commit/Abort. When a client initiates a transaction, the the

coordinator begins the preparation phase by sending a Prepare RPC to all participants. Each participant evaluates the request using its local state and responds with a vote “YES” or “NO” If all vote yes, the coordinator proceeds to the commit phase otherwise sends an Abort RPC.

5. Design of Raft Implementation

Raft algorithm initially started as followers and wait for periodic heartbeats from a leader. If a follower does not receive a heartbeat within a timeout window (1.5 to 3 seconds), it transitions into a candidate state, votes for itself, and sends RequestVote RPCs to the other nodes. Once the node receives votes from a majority, it becomes the leader and begins sending the AppendEntries RPCs as heartbeats at regular intervals to maintain authority.

As far as log replication goes, when a client submits an operation, the leader appends the request to its local log and then propagates this entry to all followers using AppendEntries RPCs. If a client mistakenly connects to a follower, the follower forwards the request to the current leader, ensuring the coordination is centralized. The followers acknowledge the replicated entries, and once the leader receives confirmations from a majority, it basically commits the entry and applies it to the state machine. The commit is reflected across all the nodes ensuring that they are all consistent.

6. Challenges and Solutions

Initial challenges such as thread deadlocks, port conflicts, and log buffer issues were identified during testing. Getting the system to pass all the tests was the challenge. But running python in unbuffered mode and changing up the logic worked.

7. Lessons Learned and Conclusion

To validate the correctness of the raft implementations, our team designed 5 test cases. The scenarios included leader crash and reelection, node rejoining with the log synchronization follower crashes while maintaining the availability, network partition handling, and split vote resolution. Each case was targeted in a specific failure mode.

The system displayed strong reliability and consistency across the scenarios we tested after implementing both the 2PC and raft algorithms.

11. References

[1] https://renjieliu.gitbooks.io/consensus-algorithms-from-2pc-to-raft/content/index.htmlLinks to an external site.

[2] https://en.wikipedia.org/wiki/Two-phase_commit_protocolLinks to an external site.

[3] Andrew S. Tanenbaum and Maarten Van Steen, Distributed Systems: Principles and Paradigms (4th Edition)

[4] https://en.wikipedia.org/wiki/Raft_(algorithm)Links to an external site.

12. Github link

PranavPujar/Project-3-DistSys: 2PC and Raft implementation on top of Group 11's gRPC Microservice arch